计算机组成与设计
习题

浙江大学 信息与电子工程学院
College of Information Science & Electronic Engineering, Zhejiang University

HW7

Problem1

Virtual memory uses a page table to track the mapping of virtual addresses to physical addresses. This exercise shows how this table must be updated as addresses are accessed. The following table is a stream of virtual addresses as seen on a system. Assume 4 KiB pages, a 4-entry fully associative TLB, and true LRU replacement. If pages must be brought in from disk, increment the next largest page number.

4095, 31272, 15789, 15000, 7193, 4096, 8912

TLB

Page Table

```
offset=12
entry=4
index=2
```

a) Given the address stream in the table, and the initial TLB and page table states shown above, show the final state of the system. Also list for each reference if it is a hit in the TLB, a hit in the page table, or a page fault.

Page size 4KiB = 2^{12} Byte

Offset => 12bits

H: Hit in TLB, M: Miss in TLB hit in page table, PF: Page Fault

addr	4095	31272	15789	15000	7193	4096	8912
Addr(0x)	0FFF	7A28	3DAD	3A98	1C19	1000	22D0
tag	0	7	3	3	1	1	2

HW7

4095	31272	15789	15000	7193	4096	8912
0	7	3	3	1	1	2
M	H	H	H	PF	H	PF

Valid	Tag	Physical Page Number
1	11	12
1	7	4
1	3	6
0	4	9

Valid	Physical page or in disk
1	5
0	Disk
0	Disk
1	6
1	9
1	11
0	Disk
1	4
0	Disk
0	Disk
1	3
1	12

4095对应的VPN=0
tag=0
Miss in TLB
page table第一行的valid=1，PPN=5
hit in page table

HW7

4095	31272	15789	15000	7193	4096	8912
0	7	3	3	1	1	2
M	H	H	H	PF	H	PF

Valid	Tag	Physical Page Number
1	11	12
1	7	4
1	3	6
1	0	5

Valid	Physical page or in disk
1	5
0	Disk
0	Disk
1	6
1	9
1	11
0	Disk
1	4
0	Disk
0	Disk
1	3
1	12

4095对应的VPN=0
tag=0
Miss in TLB
page table第一行的valid=1，PFN=5
hit in page table

浙江大学 信息与电子工程学院
College of Information Science & Electronic Engineering, Zhejiang University

HW7

4095	31272	15789	15000	7193	4096	8912
0	7	3	3	1	1	2
M	H	H	H	PF	H	PF

Valid	Tag	Physical Page Number
1	11	12
1	7	4
1	3	6
1	0	5

312272.15789.15000均hit in TLB

Valid	Physical page or in disk
1	5
0	Disk
0	Disk
1	6
1	9
1	11
0	Disk
1	4
0	Disk
0	Disk
1	3
1	12

HW7

4095	31272	15789	15000	7193	4096	8912
0	7	3	3	1	1	2
M	H	H	H	PF	H	PF

Valid	Tag	Physical Page Number
1	11	12
1	7	4
1	3	6
1	0	5

Valid	Physical page or in disk
1	5
0	Disk
0	Disk
1	6
1	9
1	11
0	Disk
1	4
0	Disk
0	Disk
1	3
1	12

7193对应的VPN=1
tag=1
Miss in TLB
page table第2行的valid=0
Page fault

HW7

4095	31272	15789	15000	7193	4096	8912
0	7	3	3	1	1	2
M	H	H	H	PF	H	PF

Valid	Tag	Physical Page Number
1	1	13
1	7	4
1	3	6
1	0	5

Valid	Physical page or in disk
1	5
1	13
0	Disk
1	6
1	9
1	11
0	Disk
1	4
0	Disk
0	Disk
1	3
1	12

7193Page fault
先载入页表
再根据LRU载入TLB

HW7

4095	31272	15789	15000	7193	4096	8912
0	7	3	3	1	1	2
M	H	H	H	PF	H	PF

Valid	Tag	Physical Page Number
1	1	13
1	7	4
1	3	6
1	0	5

4096 hit in TLB

Valid	Physical page or in disk
1	5
1	13
0	Disk
1	6
1	9
1	11
0	Disk
1	4
0	Disk
0	Disk
1	3
1	12

HW7

4095	31272	15789	15000	7193	4096	8912
0	7	3	3	1	1	2
M	H	H	H	PF	H	PF

Valid	Tag	Physical Page Number
1	1	13
1	7	4
1	3	6
1	0	5

Valid	Physical page or in disk
1	5
1	13
0	Disk
1	6
1	9
1	11
0	Disk
1	4
0	Disk
0	Disk
1	3
1	12

8912对应的VPN=2
tag=2
Miss in TLB
page table第3行的valid=0
Page fault

HW7

4095	31272	15789	15000	7193	4096	8912
0	7	3	3	1	1	2
M	H	H	H	PF	H	PF

Valid	Tag	Physical Page Number
1	1	13
1	7	4
1	3	6
1	2	14

Valid	Physical page or in disk
1	5
1	13
1	14
1	6
1	9
1	11
0	Disk
1	4
0	Disk
0	Disk
1	3
1	12

8912Page fault
先载入页表
再根据LRU载入TLB

HW7

Problem1

b) Repeat Exercise a), but this time use 16 KiB pages instead of 4 KiB pages. What would be some of the advantages of having a larger page size? What are some of the disadvantages?

Page size 4KiB = 2^{14} Byte

Offset => 14bits

H: Hit in TLB, M: Miss in TLB hit in page table, PF: Page Fault

		0111	0011	0011	0001	0001	0010
addr	4095	31272	15789	15000	7193	4096	8912
Addr(0x)	0FFF	7A28	3DAD	3A98	1C19	1000	22D0
tag	0	1	0	0	0	0	0

HW7

4095	31272	15789	15000	7193	4096	8912
0	1	0	0	0	0	0
M	PF	H	H	H	H	H

Valid	Tag	Physical Page Number
1	11	12
1	7	4
1	3	6
0	4	9

Valid	Physical page or in disk
1	5
0	Disk
0	Disk
1	6
1	9
1	11
0	Disk
1	4
0	Disk
0	Disk
1	3
1	12

4095对应的VPN=0
tag=0
Miss in TLB
page table第一行的valid=1，PFN=5
hit in page table

HW7

4095	31272	15789	15000	7193	4096	8912
0	1	0	0	0	0	0
M	PF	H	H	H	H	H

Valid	Tag	Physical Page Number
1	11	12
1	7	4
1	3	6
1	0	5

Valid	Physical page or in disk
1	5
0	Disk
0	Disk
1	6
1	9
1	11
0	Disk
1	4
0	Disk
0	Disk
1	3
1	12

4095载入TLB

HW7

4095	31272	15789	15000	7193	4096	8912
0	1	0	0	0	0	0
M	PF	H	H	H	H	H

Valid	Tag	Physical Page Number
1	11	12
1	7	4
1	3	6
1	0	5

Valid	Physical page or in disk
1	5
0	Disk
0	Disk
1	6
1	9
1	11
0	Disk
1	4
0	Disk
0	Disk
1	3
1	12

31272对应的VPN=1
tag=1
Miss in TLB
page table第2行的valid=0
Page fault

HW7

4095	31272	15789	15000	7193	4096	8912
0	1	0	0	0	0	0
M	PF	H	H	H	H	H

Valid	Tag	Physical Page Number
1	1	13
1	7	4
1	3	6
1	0	5

Valid	Physical page or in disk
1	5
1	13
0	Disk
1	6
1	9
1	11
0	Disk
1	4
0	Disk
0	Disk
1	3
1	12

31272先载入页表
再根据LRU载入TLB

HW7

4095	31272	15789	15000	7193	4096	8912
0	1	0	0	0	0	0
M	PF	H	H	H	H	H

Valid	Tag	Physical Page Number
1	1	13
1	7	4
1	3	6
1	0	5

Valid	Physical page or in disk
1	5
1	13
0	Disk
1	6
1	9
1	11
0	Disk
1	4
0	Disk
0	Disk
1	3
1	12

后续tag=0，都hit

较大的页表大小会降低TLB未命中率，但会导致更高的碎片化和物理内存的更低利用率

内部碎片（未使用页面中的所有内存）
页面错误惩罚更大（从磁盘读取的时间更长）

HW7

Problem1

c) Show the final contents of the TLB if it is 2-way set associative. Also show the contents of the TLB if it is direct mapped. Discuss the importance of having a TLB to high performance. How would virtual memory accesses be handled if there were no TLB?

Virtual page number: Address >> 12bits

tag + index = VPN

2-way set associative: 1 bits index

	4095	31272	15789	15000	7193	4096	8912
VPN	0	7	3	3	1	1	2
VPN	000	111	011	011	001	001	010
Index	0	1	1	1	1	1	0
Tag	00	11	01	01	00	00	01

HW7

	4095	31272	15789	15000	7193	4096	8912
VPN	000	111	011	011	001	001	010
Tag	00	11	01	01	00	00	01
Index	0	1	1	1	1	1	0

Index	Valid	Tag	Physical Page Number
0	1	00	5
0			
1			
1			

Valid	Physical page or in disk
1	5
0	Disk
0	Disk
1	6
1	9
1	11
0	Disk
1	4
0	Disk
0	Disk
1	3
1	12

4095hit in page table

载入TLB

HW7

	4095	31272	15789	15000	7193	4096	8912
VPN	000	111	011	011	001	001	010
Tag	00	11	01	01	00	00	01
Index	0	1	1	1	1	1	0

Index	Valid	Tag	Physical Page Number
0	1	00	5
0			
1	1	11	4
1			

Valid	Physical page or in disk
1	5
0	Disk
0	Disk
1	6
1	9
1	11
0	Disk
1	4
0	Disk
0	Disk
1	3
1	12

31272 hit in page table
载入TLB

HW7

	4095	31272	15789	15000	7193	4096	8912
VPN	000	111	011	011	001	001	010
Tag	00	11	01	01	00	00	01
Index	0	1	1	1	1	1	0

Index	Valid	Tag	Physical Page Number
0	1	00	5
0			
1	1	11	4
1	1	01	6

15789 hit in page table
载入TLB
15000 hit in TLB

Valid	Physical page or in disk
1	5
0	Disk
0	Disk
1	6
1	9
1	11
0	Disk
1	4
0	Disk
0	Disk
1	3
1	12

HW7

	4095	31272	15789	15000	7193	4096	8912
VPN	000	111	011	011	001	001	010
Tag	00	11	01	01	00	00	01
Index	0	1	1	1	1	1	0

Index	Valid	Tag	Physical Page Number
0	1	00	5
0			
1	1	00	13
1	1	01	6

Valid	Physical page or in disk
1	5
1	13
0	Disk
1	6
1	9
1	11
0	Disk
1	4
0	Disk
0	Disk
1	3
1	12

7193 page fault
先载入页表
再载入TLB（LRU）
4096 hit in TLB

HW7

	4095	31272	15789	15000	7193	4096	8912
VPN	000	111	011	011	001	001	010
Tag	00	11	01	01	00	00	01
Index	0	1	1	1	1	1	0

Index	Valid	Tag	Physical Page Number
0	1	00	5
0	1	01	14
1	1	00	13
1	1	01	6

7193 page fault
先载入页表
再载入TLB

Valid	Physical page or in disk
1	5
1	13
1	14
1	6
1	9
1	11
0	Disk
1	4
0	Disk
0	Disk
1	3
1	12

HW7

Problem1

c) Show the final contents of the TLB if it is 2-way set associative. Also show the contents of the TLB if it is direct mapped. Discuss the importance of having a TLB to high performance. How would virtual memory accesses be handled if there were no TLB?

Virtual page number: Address >> 12bits

tag + index = VPN

direct mapped: 2 bits index

	4095	31272	15789	15000	7193	4096	8912
VPN	0	7	3	3	1	1	2
VPN	000	111	011	011	001	001	010
Index	00	11	11	11	01	01	10
Tag	0	1	0	0	0	0	0

HW7

	4095	31272	15789	15000	7193	4096	8912
VPN	000	111	011	011	001	001	010
Tag	0	1	0	0	0	0	0
Index	00	11	11	11	01	01	10

Index	Valid	Tag	Physical Page Number
00	1	0	5
01			
10			
11			

Valid	Physical page or in disk
1	5
0	Disk
0	Disk
1	6
1	9
1	11
0	Disk
1	4
0	Disk
0	Disk
1	3
1	12

4095hit in page table

载入TLB

HW7

	4095	31272	15789	15000	7193	4096	8912
VPN	000	111	011	011	001	001	010
Tag	0	1	0	0	0	0	0
Index	00	11	11	11	01	01	10

Index	Valid	Tag	Physical Page Number
00	1	0	5
01			
10			
11	1	1	4

Valid	Physical page or in disk
1	5
0	Disk
0	Disk
1	6
1	9
1	11
0	Disk
1	4
0	Disk
0	Disk
1	3
1	12

31272hit in page table

载入TLB

HW7

	4095	31272	15789	15000	7193	4096	8912
VPN	000	111	011	011	001	001	010
Tag	0	1	0	0	0	0	0
Index	00	11	11	11	01	01	10

Index	Valid	Tag	Physical Page Number
00	1	0	5
01			
10			
11	1	0	6

Valid	Physical page or in disk
1	5
0	Disk
0	Disk
1	6
1	9
1	11
0	Disk
1	4
0	Disk
0	Disk
1	3
1	12

15789 hit in page table
载入TLB
Index=11，替换
15000 直接hit in TLB

HW7

	4095	31272	15789	15000	7193	4096	8912
VPN	000	111	011	011	001	001	010
Tag	0	1	0	0	0	0	0
Index	00	11	11	11	01	01	10

Index	Valid	Tag	Physical Page Number
00	1	0	5
01	1	0	13
10			
11	1	0	6

Valid	Physical page or in disk
1	5
1	13
0	disk
1	6
1	9
1	11
0	Disk
1	4
0	Disk
0	Disk
1	3
1	12

7193 page fault
载入页表
载入TLB
4096 hit in TLB

HW7

	4095	31272	15789	15000	7193	4096	8912
VPN	000	111	011	011	001	001	010
Tag	0	1	0	0	0	0	0
Index	00	11	11	11	01	01	10

Index	Valid	Tag	Physical Page Number
00	1	0	5
01	1	0	13
10	1	0	14
11	1	0	6

Valid	Physical page or in disk
1	5
1	13
1	14
1	6
1	9
1	11
0	Disk
1	4
0	Disk
0	Disk
1	3
1	12

8912 page fault

载入页表

载入TLB

HW7

c) Discuss the importance of having a TLB to high performance. How would virtual memory accesses be handled if there were no TLB?

The TLB is important to avoiding paying high access times to memory in order to translate virtual addresses to physical addresses. If memory accesses are frequent, then the TLB will become even more important.

Without a TLB, the page table would have to be referenced upon every access using a virtual addresses, causing a significant slowdown.

TLB对于避免为将虚拟地址转换为物理地址而花费大量内存访问时间非常重要。如果内存访问频繁，那么TLB将变得更加重要。

如果没有TLB，每次访问时都必须使用虚拟地址引用页表，这会导致显著的速度减慢。

HW7

There are several parameters that impact the overall size of the page table. Listed below are several key page table parameters.

Virtual address size	Page size	Page table entry size
32 bits	4 KiB	8 bytes

d) Given the parameters in the table above, calculate the total page table size for a system running 5 applications that utilize half of the memory available.

The tag size is 32 - $\log_2(4 * 1024)$ = 32 - 12 = 20 bits.

All five page tables would require 5 * 2^{20} *8 bytes = 40 MiB

页表一定要覆盖所有虚拟地址空间

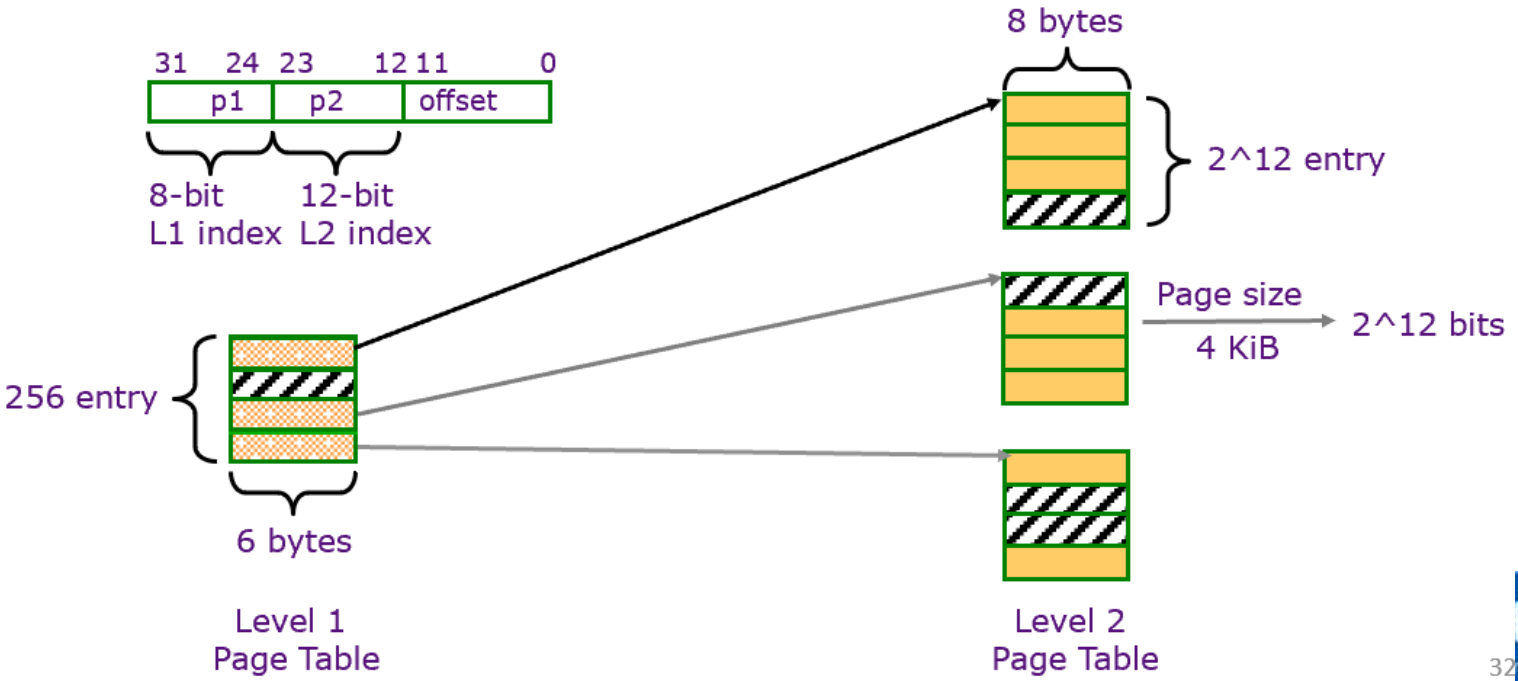
2^32/2^12*8*5=2^23*5=40MB

HW7

e) Calculate the total page table size for a system running 5 applications that utilize half of the memory available, given a two-level page table approach with 256 entries. Assume each entry of the main page table is 6 bytes. Calculate the minimum and maximum amount of memory required.

The 2^{20} page table entries are divided into 256 segments.

Each of the second-level tables contain $2^{20-8} = 2^{12}$entries, requiring 2^{12}*8B=32 KiB each and covering 2^{12}*4 KiB=16 MiB (2^{24}) of the virtual address space.



HW7

e) Calculate the total page table size for a system running 5 applications that utilize half of the memory available, given a two-level page table approach with 256 entries. Assume each entry of the main page table is 6 bytes. Calculate the minimum and maximum amount of memory required.

The 2^{20} page table entries are divided into 256 segments.

Each of the second-level tables contain $2^{20-8} = 2^{12}$entries, requiring 2^{12}*8B=32 KiB each and covering 2^{12}*4 KiB=16 MiB (2^{24}) of the virtual address space.

Min: first-level 5 * 256 * 6 bytes = 7680 bytes

second-level 5 * (2^{31} / 2^{24}) * 32KiB = 20MiB

Max(all segments were activated):

first-level 5 * 256 * 6 bytes = 7680 bytes

second-level 5 * (2^{32} / 2^{24}) * 32KiB = 40MiB

HW7

f) A cache designer wants to increase the size of a 4 KiB virtually indexed, physically tagged cache. Given the page size listed in the table above, is it possible to make a 16 KiB direct-mapped cache, assuming 2 words per block? How would the designer increase the data size of the cache?

2 words per blocks => 8 bytes/block => 3 bits offset

16 KiB/(8 bytes/block) => 2^{11} sets => 11 bits index

With a 4 KiB page, the lower 12 bits are available for use for indexing prior to translation from VA to PA. However, a 16 KiB Direct-Mapped cache needs the lower 14 bits to remain the same. Thus, it is not possible to build this cache.

A higher associativity must be used. If the cache has 2 words per block, only 9 bits are available for indexing (thus 512 sets). To make a 16 KiB cache, a 4-way associativity must be used.

HW7

Problem2

Consider a system with 40-bit virtual addresses, 36-bit physical addresses, and 64 KiB (2^{16} bytes) pages. The system uses a page map to translate virtual addresses to physical addresses; each page map entry includes dirty (D) and resident (R) bits.

a) Assuming a flat page map, what is the size of each page map entry, and how many entries does the page map have?

offset： 16 bits

tag： 36 – 16 = 20 bits

entry： 20 + 1 + 1 = 22 bits

Size of page map entry in bits: 22 bits

Number of entries in the page map: $2^{40-16} = 2^{24}$

HW7

Problem2

Consider a system with 40-bit virtual addresses, 36-bit physical addresses, and 64 KiB (2^{16} bytes) pages. The system uses a page map to translate virtual addresses to physical addresses; each page map entry includes dirty (D) and resident (R) bits.

b) If changed the system to use 16 KiB (2^{14} bytes) pages instead of 64 KiB pages, how would the number of entries in the page map change? Please give the ratio of the new size to the old size.

(# entries with 16 KiB pages) / (# entries with 64 KiB pages):

$2^{40-14} / 2^{40-16} = 4$

HW7

Problem2

c) Assume we still use 64 KiB pages. The contents of the page map and TLB are shown to the right. The page map uses an LRU replacement policy and the LRU page (shown below) will be chosen for replacement. For each of these four accesses, compute its corresponding physical address and indicate whether the access causes a TLB miss and/or a page fault. Assume each access starts with the TLB and Page Map state shown to the right.

Offset=16

Virtual Addr	PPN(in hex)	PhysAddr(in Hex)	TLB Miss?(Y/N)	Page Fault?(Y/N)
0x06004				
0x30286				
0x68030				
0x4BEEF				

HW7

VPN (tag)	V	D	PPN
0x0	1	0	0xBE7A
0x3	0	0	0x7
0x5	1	1	0xFF
0x2	1	0	0x900

VPN	R	D	PPN	
0	1	0	0xBE7A	
1	0	0	---	
2	1	0	0X900	
3	1	0	0x8	LRU PAGE
4	0	0	---	
5	1	1	0xFF	
6	1	0	0x70	

Virtual Addr	PPN(in hex)	PhysAddr(in Hex)	TLB Miss?(Y/N)	Page Fault?(Y/N)
0x06004	BE7A	BE7A6004	N	N
0x30286				
0x68030				
0x4BEEF				

HW7

VPN (tag)	V	D	PPN
0x0	1	0	0xBE7A
0x3	0	0	0x7
0x5	1	1	0xFF
0x2	1	0	0x900

VPN	R	D	PPN
0	1	0	0xBE7A
1	0	0	---
2	1	0	0X900
3	1	0	0x8
4	0	0	---
5	1	1	0xFF
6	1	0	0x70

LRU PAGE

Virtual Addr	PPN(in hex)	PhysAddr(in Hex)	TLB Miss?(Y/N)	Page Fault?(Y/N)
0x06004	BE7A	BE7A6004	N	N
0x30286	8	80286	Y	N
0x68030				
0x4BEEF				

HW7

VPN (tag)	V	D	PPN
0x0	1	0	0xBE7A
0x3	1	1	0x8
0x5	1	1	0xFF
0x2	1	0	0x900

VPN	R	D	PPN
0	1	0	0xBE7A
1	0	0	---
2	1	0	0X900
3	1	0	0x8
4	0	0	---
5	1	1	0xFF
6	1	0	0x70

LRU PAGE

Virtual Addr	PPN(in hex)	PhysAddr(in Hex)	TLB Miss?(Y/N)	Page Fault?(Y/N)
0x06004	BE7A	BE7A6004	N	N
0x30286	8	80286	Y	N
0x68030	70	708030	Y	N
0x4BEEF				

HW7

VPN (tag)	V	D	PPN
0x0	1	0	0xBE7A
0x3	1	1	0x8
0x5	1	1	0xFF
0x6	1	0	0x70

VPN	R	D	PPN	
0	1	0	0xBE7A	
1	0	0	---	
2	1	0	0X900	
3	1	0	0x8	LRU PAGE
4	0	0	---	
5	1	1	0xFF	
6	1	0	0x70	

Virtual Addr	PPN(in hex)	PhysAddr(in Hex)	TLB Miss?(Y/N)	Page Fault?(Y/N)
0x06004	BE7A	BE7A6004	N	N
0x30286	8	80286	Y	N
0x68030	70	708030	Y	N
0x4BEEF			Y	Y

HW8

Problem1

a. Why is the cost of a page fault in virtual memory considered very high?
b. What are the typical advantages of having larger page size?
c. Write one advantage and one disadvantage of a virtually tagged cache?
d. Which type of parallel computing architecture is no longer commonly encountered in machines today (MIMD, MISD, SIMD, SISD)?

- 为什么虚拟内存中的页面错误的成本被认为非常高？
  - 发生page fault需要访问disk，访问时间长
- 拥有更大page size的典型优势是什么？
  - 缓解 TLB 压力
  - 降低页表开销
  - 消除页表查找开销。
- 虚拟标记缓存的优缺点
  - 虚拟地址标记，查找速度快。
  - 会产生cache aliasing问题，导致cache中数据不一致的问题
- 哪种类型的并行计算架构在当今的机器中不再常见（MIMD、MISD、SIMD、SISD）？
  - MISD

HW8

Problem 2

An important advantage of interrupts over polling is the ability of the processor to perform other tasks while waiting for communication from an I/O device. Suppose that a 2GHz processor needs to read 1000 bytes of data from a particular I/O device. The I/O device supplies 1 bytes of data every 0.01ms. **The code to process the data and store it in a buffer takes 2000 cycles.**

2000cycles 针对的是 1000 bytes 的数据。

a. If the processor detects that a byte of data is ready through polling, and a polling iteration takes 40 cycles, how many cycles does the entire operation take?

b. instead, the processor is interrupted when a byte is ready, and the processor spends the time between interrupts on another task, how many cycles of this other task can the processor complete while the I/O communication is taking place? The overhead for handling an interrupt is 100 cycles.

HW8

(a) 假定处理器在下一个byte前会处理数据，处理器的频率为2GHz，I/O 每0.01ms 提供一个byte，即一个byte需要的时间为：

0.01ms*2GHz=20000cycles

而处理一个byte的时间则为2000/1000=2cycles，可忽略，一个polling iteration 的时间为40cycles，所以0.01ms，共

200000/40=500 polls

检测一个新的byte，需要一整个polling iteration，所以1个byte 需要500*40=20000cycles

总共1000byte，需要20000*1000=20000000 cycles = $2*10^7$ cycles

(b) 每一次处理器需要100cycles来处理中断，因此对于1byte来说有

0.01ms*2GHz-100cycles=19900 cycles 来处理其他任务

对于1000 bytes 有19900*1000=19900000 cycles for other task